

从 Vector Add 到 Tiled GEMM

GPU Programming

陈楷骐

ZJUSCT

2026-07-13

01 **SIMT GPU Architecture**

SIMT GPU 架构和 CUDA 编程模型

02 **Tensor Core: From Volta to Blackwell**

Tensor Core 的演进

03 **Data Movement and Layout**

数据搬运和排布

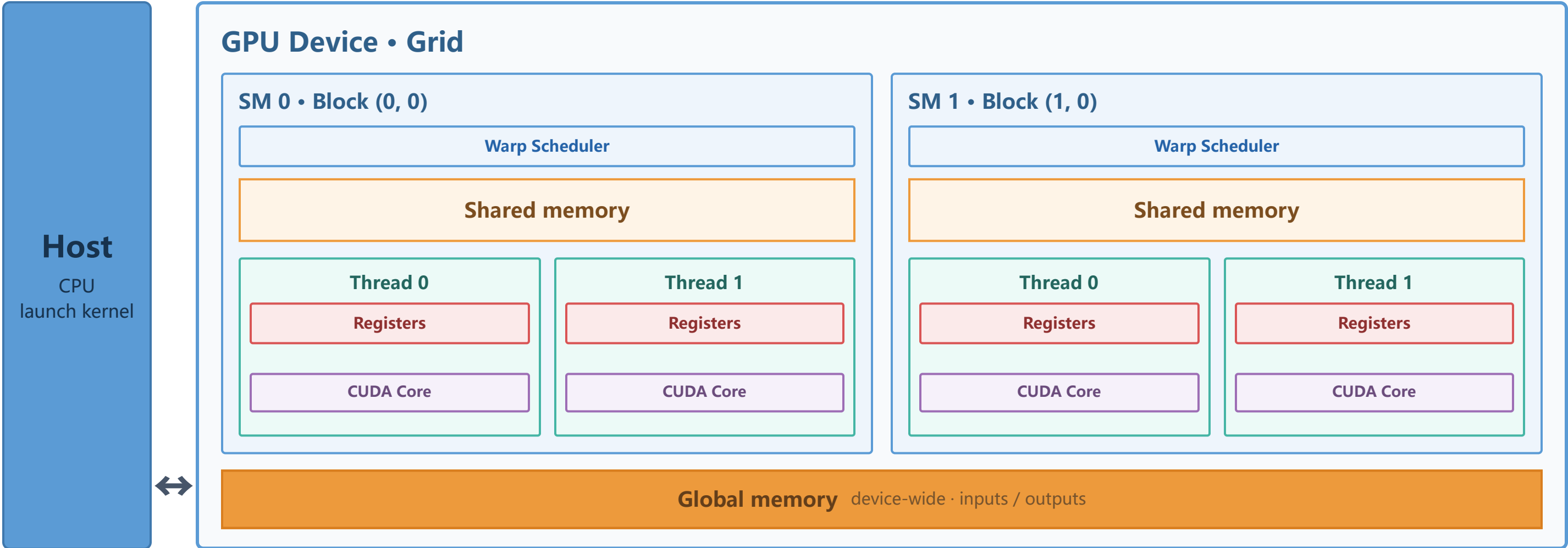
04 **Tiled GEMM in TileLang**

现代 GPU 编程

SIMT GPU Architecture

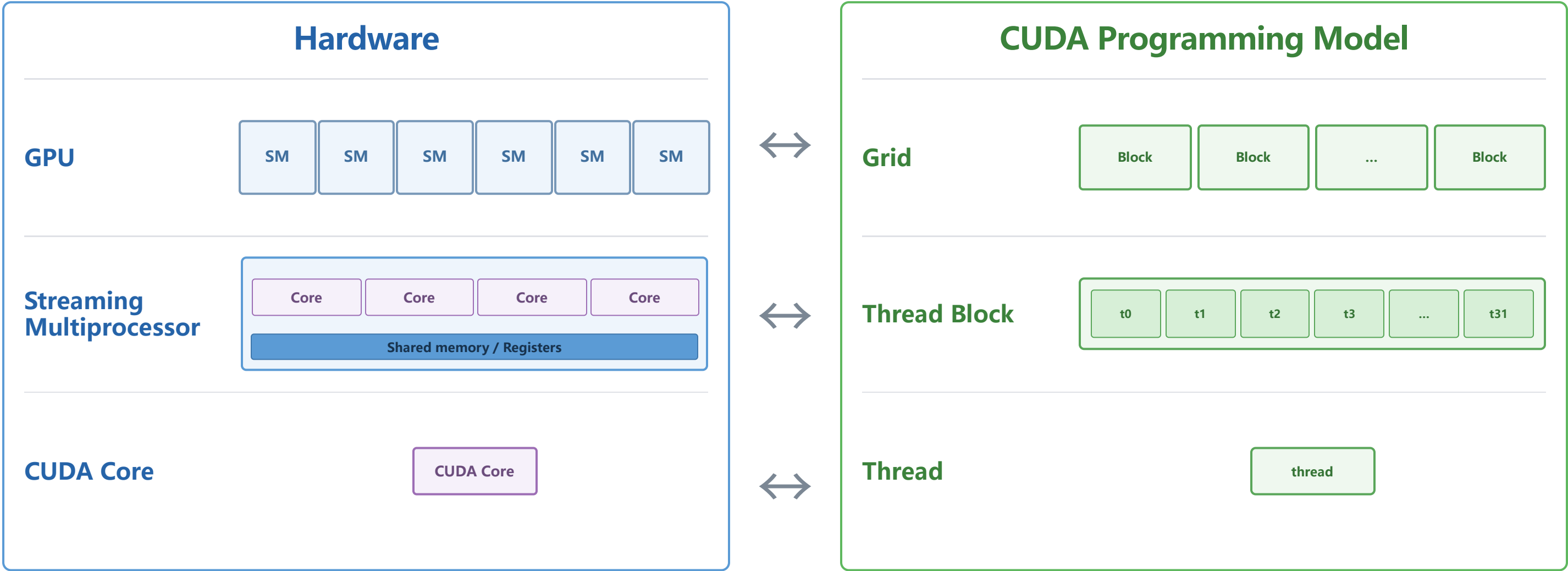
SIMT GPU 架构和 CUDA 编程模型

SIMT GPU 架构



SIMT = Single Instruction, Multiple Threads: 多个 Thread (32 Threads(lanes) = 1 Warp) 共享指令流, 但各自保留寄存器和数据。

硬件架构 ↔ CUDA 编程模型



GPU 负责承载整个 Kernel 的 Grid
SM 是 Threadblock 的执行位置
CUDA Core 执行单个 Thread 的算术指令

Example: Vector Add

考虑将 CPU 上的串行 Vector Add 移植到 GPU 上

CPU Serial Program

```
1 for (int i = 0; i < N; ++i) {  
2     C[i] = A[i] + B[i];  
3 }
```

CPU 串程序序

一个 thread 依次取得 $i = 0, 1, \dots, N - 1$ 。

Device Kernel

```
1 __global__ void vector_add(  
2     const float* A,  
3     const float* B,  
4     float* C,  
5     int N  
6 ) {  
7     int i = blockIdx.x * blockDim.x + threadIdx.x;  
8     if (i < N) {  
9         C[i] = A[i] + B[i];  
10    }  
11 }
```

GPU Kernel

同一个 Kernel 被许多 Thread 同时执行, 每个 Thread 计算自己应该处理的元素。

设置 Grid/Block 大小

接下来是如何启动 GPU Kernel

Device code

```
1 __global__ void vector_add(  
2     const float* A, const float* B,  
3     float* C, int N  
4 ) {  
5     int i = blockIdx.x * blockDim.x + threadIdx.x;  
6     int stride = blockDim.x * gridDim.x;  
7  
8     for (; i < N; i += stride) {  
9         C[i] = A[i] + B[i];  
10    }  
11 }
```

考虑一个 thread 处理多个 element

`stride = blockDim.x * gridDim.x`, 每个 thread 依次处理 `i`、`i + stride`、`i + 2 × stride`。

Host code

```
1 int threads = 256;  
2 int block_stride = threads * elem_per_thread;  
3 int blocks = ceil_div(N, block_stride);  
4  
5 vector_add<<<blocks, threads>>>(A, B, C, N);  
6 // or use fixed grid size  
7 vector_add<<<64, 256>>>(A, B, C, N);
```

CPU Launch Kernel

`<<<blocks, threads>>>` 定义 grid 和 block 的大小, 然后提交 kernel。

完整的 CUDA 程序

Device code

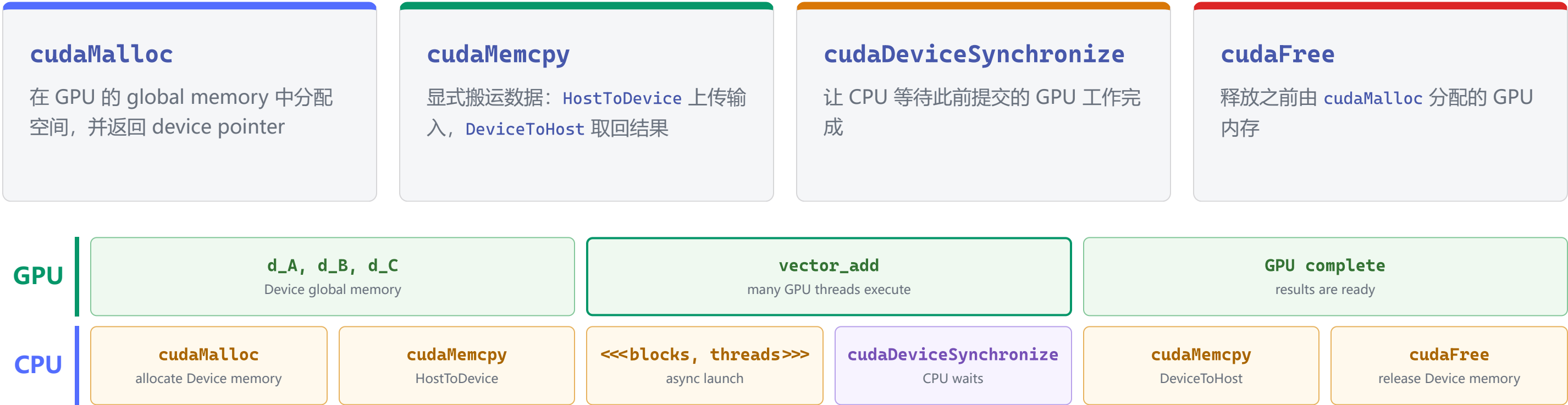
```
1 __global__ void vector_add(  
2     const float* A,  
3     const float* B,  
4     float* C,  
5     int N  
6 ) {  
7     int i = blockIdx.x * blockDim.x + threadIdx.x;  
8     int stride = blockDim.x * gridDim.x;  
9     for (; i < N; i += stride) C[i] = A[i] + B[i];  
10 }
```

Host code

```
1 #include <cuda_runtime.h>  
2 #include <cstdio>  
3 #include <vector>  
4  
5 int main() {  
6     const int N = 1 << 20;  
7     const size_t bytes = size_t(N) * sizeof(float);  
8     std::vector<float> h_A(N, 1.0f), h_B(N, 2.0f), h_C(N);  
9     float *d_A = nullptr, *d_B = nullptr, *d_C = nullptr;  
10    cudaMalloc(&d_A, bytes);  
11    cudaMalloc(&d_B, bytes);  
12    cudaMalloc(&d_C, bytes);  
13    cudaMemcpy(d_A, h_A.data(), bytes, cudaMemcpyHostToDevice);  
14    cudaMemcpy(d_B, h_B.data(), bytes, cudaMemcpyHostToDevice);  
15    vector_add<<<64, 256>>>(d_A, d_B, d_C, N);  
16    cudaDeviceSynchronize();  
17    cudaMemcpy(h_C.data(), d_C, bytes, cudaMemcpyDeviceToHost);  
18    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);  
19    return 0;  
20 }
```


CUDA Runtime API

这些 API 把 Host 内存、Device 内存和异步 kernel 串成一条执行路径。

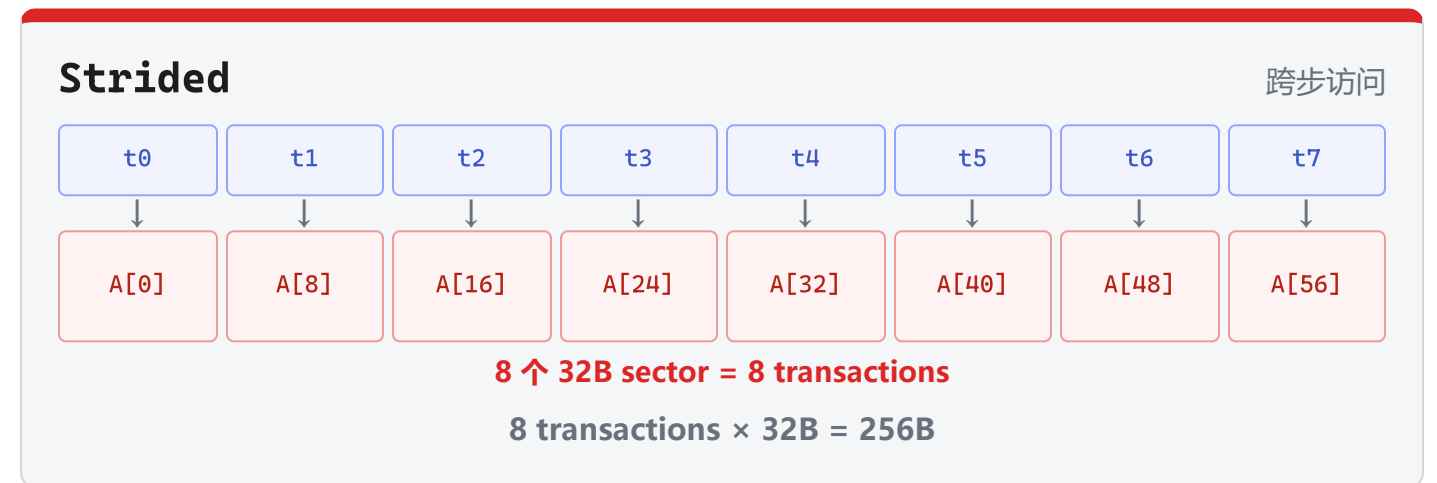
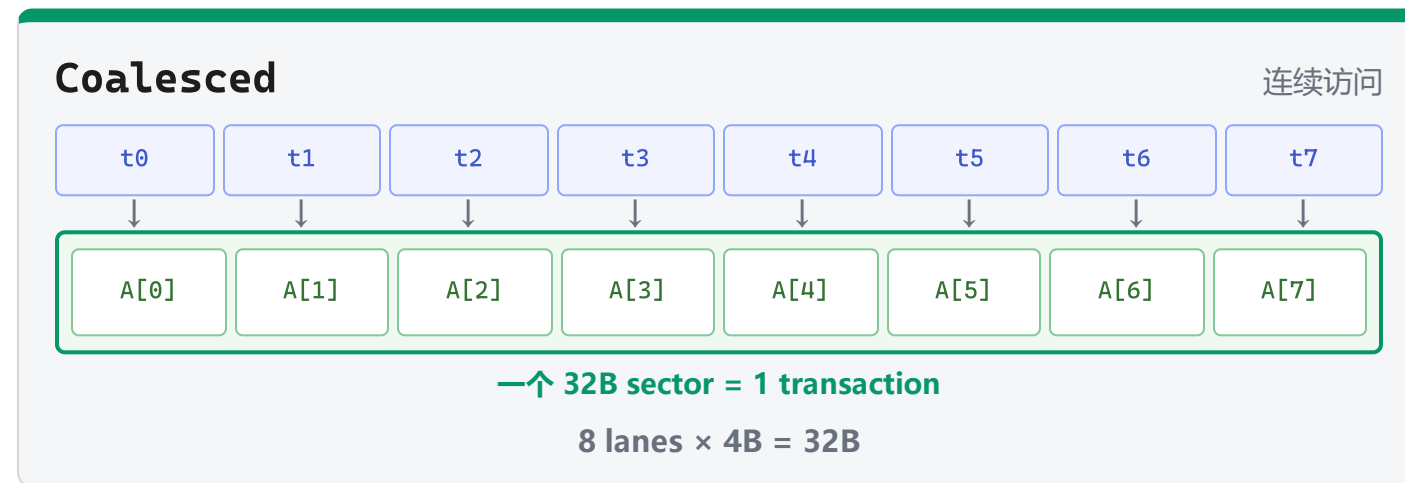


Kernel launch 默认是异步的：CPU 提交工作后可以继续运行；只有在同步或数据回拷等边界处，才明确等待 GPU 完成。

Memory Coalescing

考虑每个 Thread 访问一个 `int = 4B`

Global Memory 一次会搬运连续的 **32B sector**, 称为一个 Transaction



让一个 Warp 访问连续的地址

保证相邻 Thread 处理相邻的任务 `A[base + threadIdx.x]`

避免一个 Warp 访问不连续的地址

例如访问按行存储的矩阵的一列 `A[base + threadIdx.x * stride]`

Warp Divergence

一个 Warp 共享指令流；当 lanes 选择不同路径时，所有路径需要依次执行

```
if (threadIdx.x < 16) path_A(); else path_B();
```

同一个 warp 内出现两种选择

Path A: 先执行

t0

t1

t2

t3

t4

t5

t6

t7

一半 active, 一半 masked

Path B: 再执行

t0

t1

t2

t3

t4

t5

t6

t7

另一半 lane 被重新激活，完成另一条路径

分支一致

如果整个 Warp 都走同一条路径，效率最高

产生分歧

每条路径都要执行，一个 Warp 要被反复执行多次，效率下降，

本节小结

从 SIMT 硬件、CUDA 编程模型到一个完整的 Vector Add 程序

1 SIMT Hardware

GPU 由多个 SM 组成, Warp 共享指令流, Thread 保留自己的寄存器

2 CUDA Programming Model

Grid、Block、Warp 和 Thread 把问题按层级拆分

3 CUDA Runtime

cudaMalloc/cudaFree、cudaMemcpy、kernel launch、同步

4 Optimization

注意 Memory Coalescing 和避免 Warp Divergence

下一步：考虑更复杂的 Tiled GEMM 和 Tensor Core

Tensor Core

从 Tiled GEMM 开始

Recall Tiled GEMM

Round 6: Cut M , N , or K ?

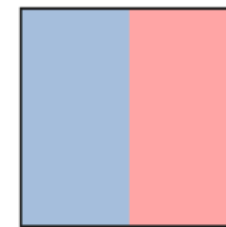
Every parallel GEMM cuts $M \times N \times K$ — **R5 already cut M** . Three choices:



cut M : strips of C



cut $M+N$: tiles of C



$\times$



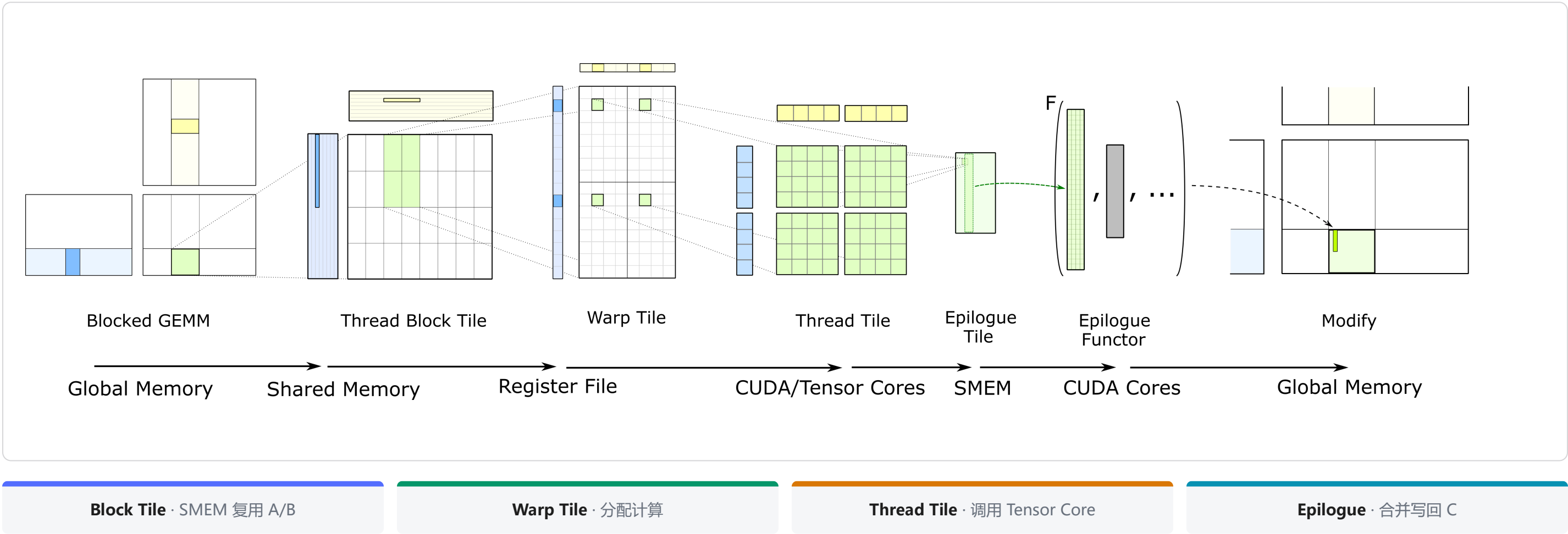
cut K : partial C s $\rightarrow$ **reduce**

- M/N : disjoint C , no reduction: by shared L3
- K : private C s and reduction.

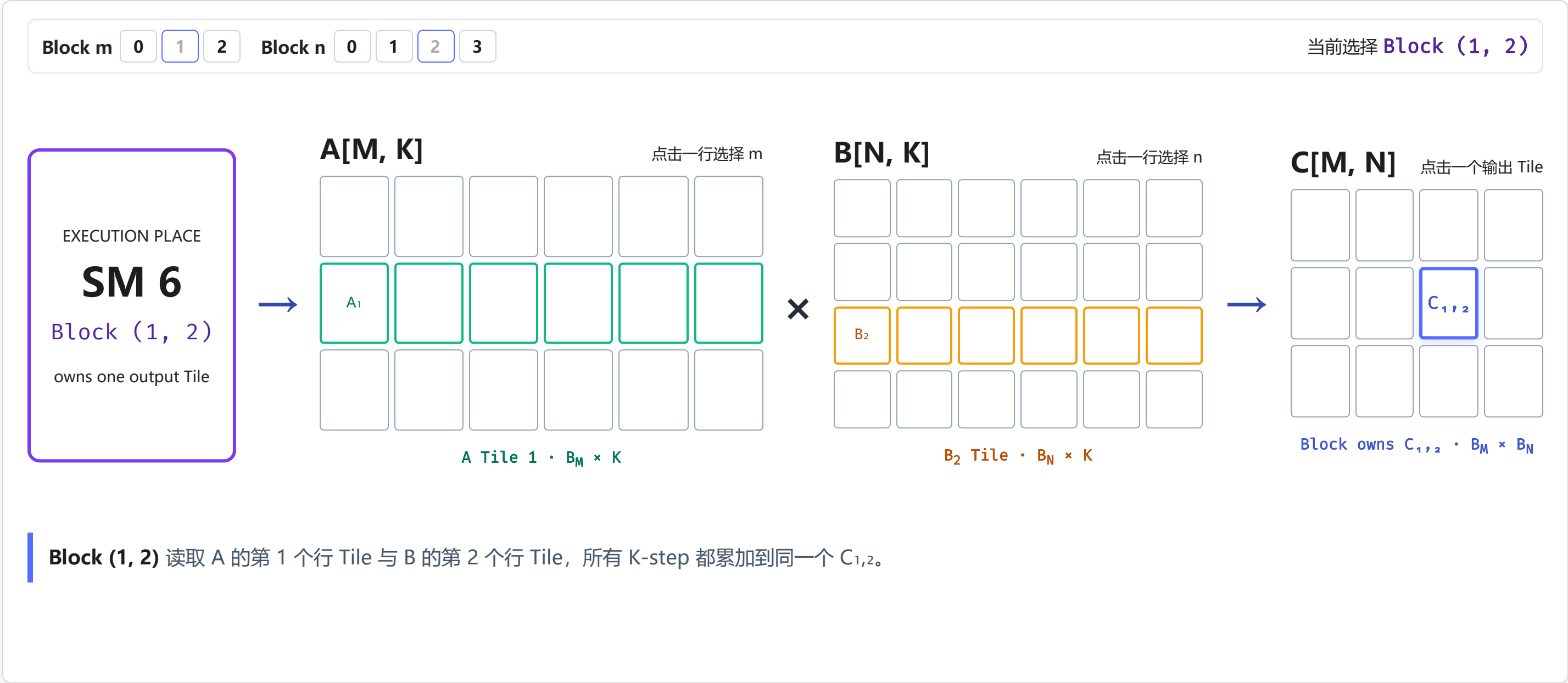
GPU Tiled GEMM Hierarchy

和 CPU 上的多核 Tiled GEMM 非常类似：一个 Block 拥有一个 C Tile，并被调度到某个 SM 上执行

核心区别在于 Shared Memory 和 Tensor Core 需要特殊的编程

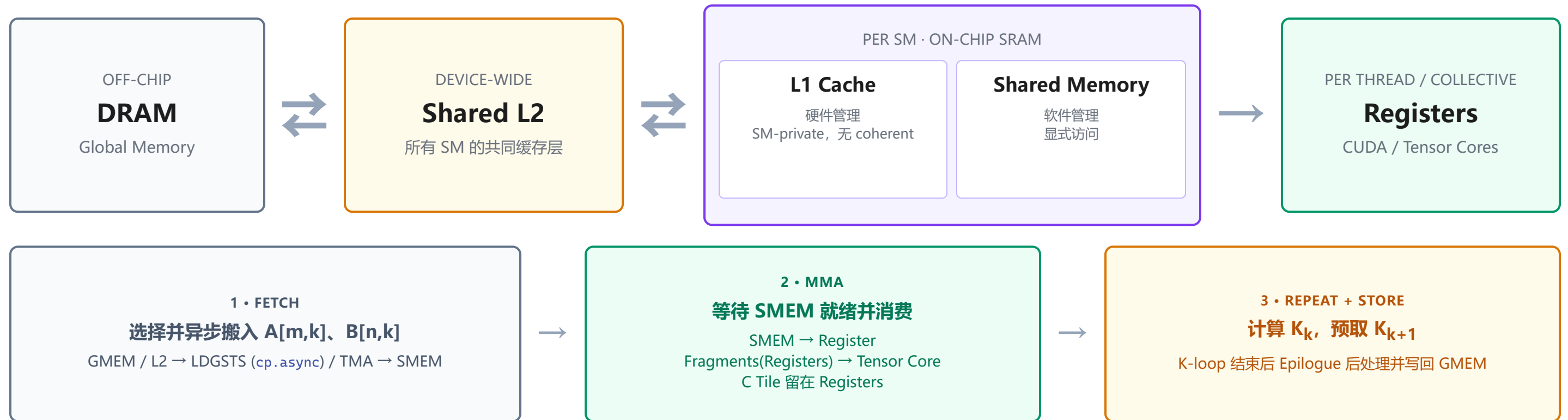


一个 Block 对应一个 C Tile



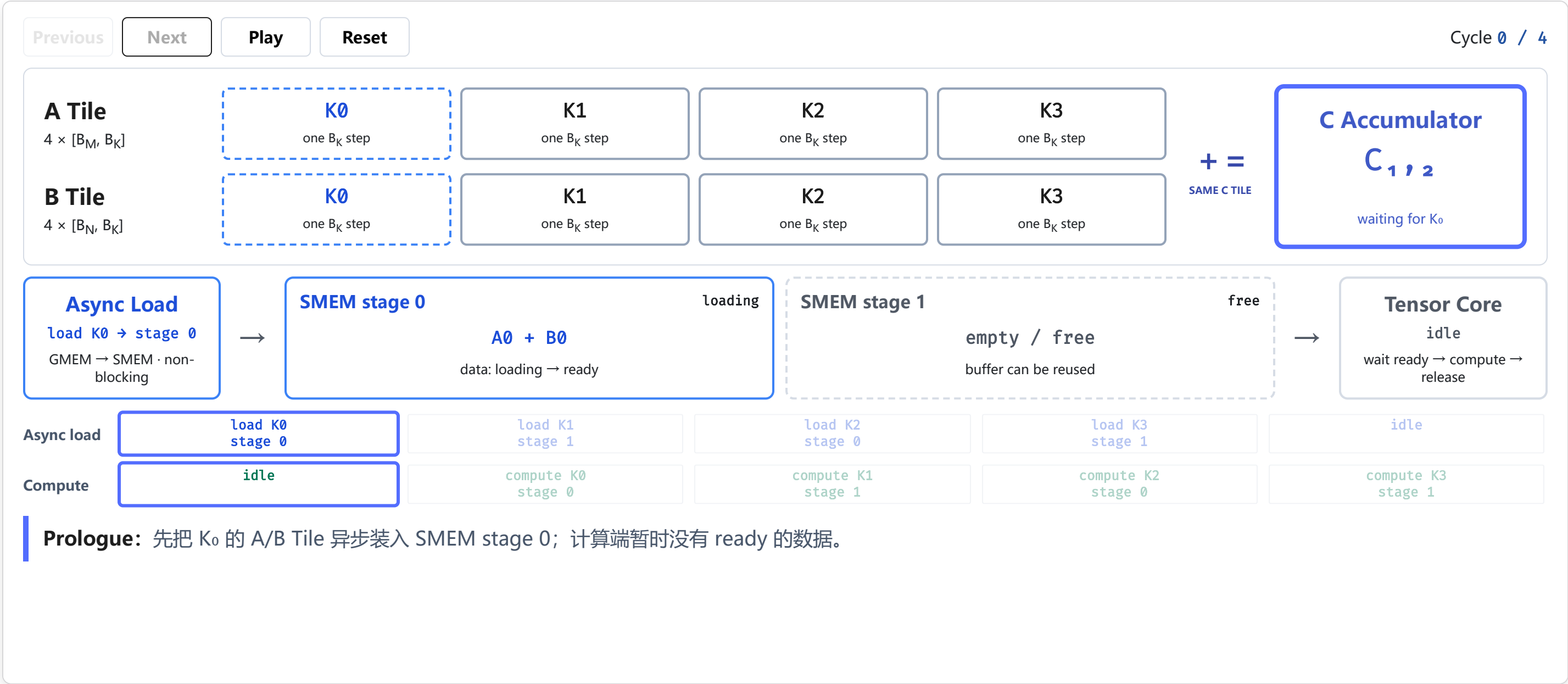
L2/L1 Cache 与 Shared Memory

固定 Block (m,n) 后, A/B Tile 向计算核心流动; C Tile 在寄存器中沿 K 持续累加。



L1 Cache 自动工作, Shared Memory 由程序显式管理。用户可以手动控制片上高速 SRAM 来处理大量输入。

沿 K 方向迭代：更新同一个 C Tile



THIS SECTION

Tensor Core Design

一个 Warp 怎样完成一次 MMA?

Instruction shape

Fragment Mapping

Volta/Ampere
Hopper
Blackwell



NEXT SECTION

Data Movement Design

怎样持续把下一组 A/B Tile 喂给它?

GMEM → SMEM
SMEM → Register

Layout / bank conflict

Async pipeline

Volta / Ampere Tensor Core: MMA

Tensor Core 指令 mma: 一个 Warp 同步调用, 完成一个 $(M, N, K) = (16, 8, 8)$ 的小型 $D = A \times B + C$ 矩阵乘法, 输入和输出均在寄存器里(Fragment)。

```
mma.sync.aligned.m16n8k8.row.col.f32.f16.f16.f32
{d0,d1,d2,d3}, {a0,a1,a2,a3}, {b0,b1}, {c0,c1,c2,c3};
```

SM80 • $16 \times 8 \times 8$

1. 选择 Thread

warp lane 0-31

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

2. 选择逻辑寄存器

operand slot

A	a0	a1	a2	a3
B	b0	b1		
C/D	c0	c1	c2	c3

Thread 0 • a0
 $A[0,0]$

$A \cdot 16 \times 8$

row major

T0V0	T0V1	T1V0	T1V1	T2V0	T2V1	T3V0	T3V1
T4V0	T4V1	T5V0	T5V1	T6V0	T6V1	T7V0	T7V1
T8V0	T8V1	T9V0	T9V1	T10V0	T10V1	T11V0	T11V1
T12V0	T12V1	T13V0	T13V1	T14V0	T14V1	T15V0	T15V1
T16V0	T16V1	T17V0	T17V1	T18V0	T18V1	T19V0	T19V1
T20V0	T20V1	T21V0	T21V1	T22V0	T22V1	T23V0	T23V1
T24V0	T24V1	T25V0	T25V1	T26V0	T26V1	T27V0	T27V1
T28V0	T28V1	T29V0	T29V1	T30V0	T30V1	T31V0	T31V1
T0V2	T0V3	T1V2	T1V3	T2V2	T2V3	T3V2	T3V3
T4V2	T4V3	T5V2	T5V3	T6V2	T6V3	T7V2	T7V3
T8V2	T8V3	T9V2	T9V3	T10V2	T10V3	T11V2	T11V3
T12V2	T12V3	T13V2	T13V3	T14V2	T14V3	T15V2	T15V3
T16V2	T16V3	T17V2	T17V3	T18V2	T18V3	T19V2	T19V3
T20V2	T20V3	T21V2	T21V3	T22V2	T22V3	T23V2	T23V3
T24V2	T24V3	T25V2	T25V3	T26V2	T26V3	T27V2	T27V3
T28V2	T28V3	T29V2	T29V3	T30V2	T30V3	T31V2	T31V3

×

$B \cdot 8 \times 8$

column major

T0V0	T4V0	T8V0	T12V0	T16V0	T20V0	T24V0	T28V0
T0V1	T4V1	T8V1	T12V1	T16V1	T20V1	T24V1	T28V1
T1V0	T5V0	T9V0	T13V0	T17V0	T21V0	T25V0	T29V0
T1V1	T5V1	T9V1	T13V1	T17V1	T21V1	T25V1	T29V1
T2V0	T6V0	T10V0	T14V0	T18V0	T22V0	T26V0	T30V0
T2V1	T6V1	T10V1	T14V1	T18V1	T22V1	T26V1	T30V1
T3V0	T7V0	T11V0	T15V0	T19V0	T23V0	T27V0	T31V0
T3V1	T7V1	T11V1	T15V1	T19V1	T23V1	T27V1	T31V1

=

$C/D \cdot 16 \times 8$

accumulator

T0V0	T0V1	T1V0	T1V1	T2V0	T2V1	T3V0	T3V1
T4V0	T4V1	T5V0	T5V1	T6V0	T6V1	T7V0	T7V1
T8V0	T8V1	T9V0	T9V1	T10V0	T10V1	T11V0	T11V1
T12V0	T12V1	T13V0	T13V1	T14V0	T14V1	T15V0	T15V1
T16V0	T16V1	T17V0	T17V1	T18V0	T18V1	T19V0	T19V1
T20V0	T20V1	T21V0	T21V1	T22V0	T22V1	T23V0	T23V1
T24V0	T24V1	T25V0	T25V1	T26V0	T26V1	T27V0	T27V1
T28V0	T28V1	T29V0	T29V1	T30V0	T30V1	T31V0	T31V1
T0V2	T0V3	T1V2	T1V3	T2V2	T2V3	T3V2	T3V3
T4V2	T4V3	T5V2	T5V3	T6V2	T6V3	T7V2	T7V3
T8V2	T8V3	T9V2	T9V3	T10V2	T10V3	T11V2	T11V3
T12V2	T12V3	T13V2	T13V3	T14V2	T14V3	T15V2	T15V3
T16V2	T16V3	T17V2	T17V3	T18V2	T18V3	T19V2	T19V3
T20V2	T20V3	T21V2	T21V3	T22V2	T22V3	T23V2	T23V3
T24V2	T24V3	T25V2	T25V3	T26V2	T26V3	T27V2	T27V3
T28V2	T28V3	T29V2	T29V3	T30V2	T30V3	T31V2	T31V3

Thread 0 的 a0: 表示 $A[0,0]$ 这个元素。

Hopper Tensor Core: WGMMA

4 个 Warp 同步发起 Tensor Core 然后异步执行

```
wgmma.mma_async.sync.aligned.m64n256k16.f32.f16.f16 d, a-desc, b-desc, (ctrl-imms);
```

SM90 • 64 × 256 × 16

WarpGroup: 128 Threads

Warp 0 • 32

Warp 1 • 32

Warp 2 • 32

Warp 3 • 32

.sync.aligned: 集体发起, 不代表计算完成

异步执行

fence

mma_async

commit

WarpGroup
继续准备下一批工作

Tensor Core
WGMMA running...

wait_group → consume C/D

类比: cpu launch kernel → gpu runs → cpu synchronize

Tensor Core 可以直接从 SMEM 读入

A
SMEM / Registers

B
SMEM

WGMMA Engine

C / D Registers

把 A/B 输入从 Register File 中移走

Volta/Ampere: `mma.sync` 依赖 SIMT 和 scoreboard 隐藏延迟

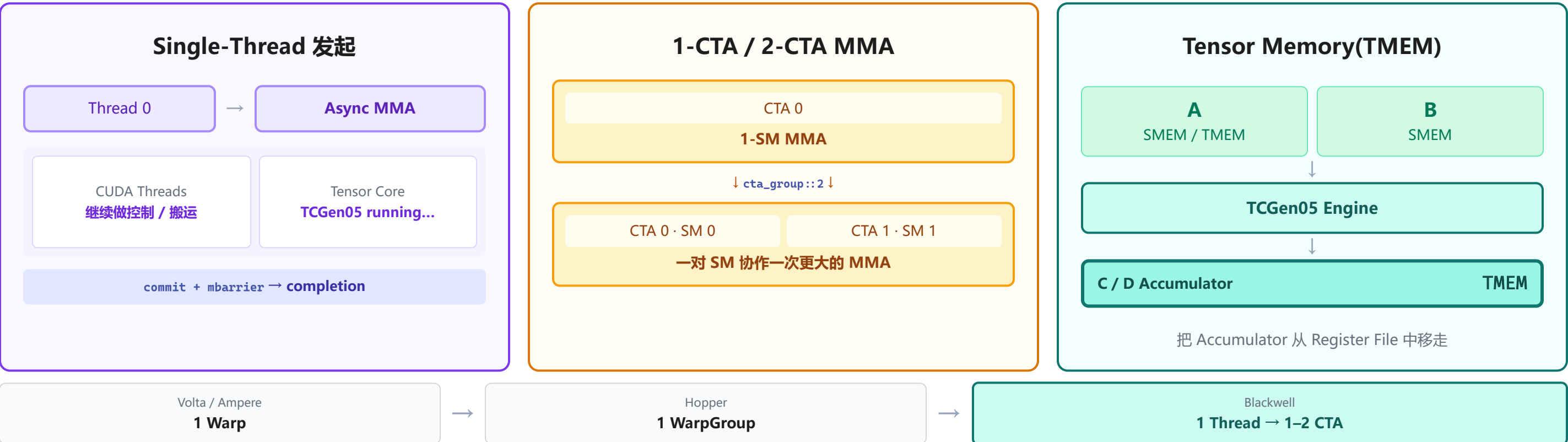
Hopper: 异步执行, 显式等待; 输入寄存器压力下降, 支持更大的 Tile

Blackwell Tensor Core: TCGen05 + TMEM

Accumulator 离开通用寄存器；单个 Thread 即可启动 1-CTA 或 2-CTA 异步 MMA。

```
tcgen05.mma.cta_group:: {1|2} [d-tmem], a-desc, b-desc, idesc, (ctrl-imms);
```

5th Gen Tensor Core



SIMT 仍是 CUDA 的控制基础；但 Tensor Core 的核心执行已经从逐 Lane 通用计算，演进为专用、异步、可独立调度的加速器。

Data Movement and Layout

数据搬运和排布

从 HBM 到 Tensor Core

我们先以 Ampere(SM80) 为例，会涉及到两次搬运。

1 • GMEM → SMEM

Global Memory (HBM) → Shared Memory

- Block 协作搬入下一组 A/B Tile
- 合并 GMEM 访问，并用 `cp.async` 隐藏 HBM 延迟
- 等待数据 ready 后，SMEM stage 才能被消费

2 • SMEM → Register

Shared Memory → Register Fragments

- Warp 使用 `ldmatrix` 协作读取 A/B Tile
- Register fragments 随后交给 `mma.sync`

首先考虑 SMEM → Register 这一步。

ldmatrix: SMEM → Register Fragment

AMPERE • WARP-COLLECTIVE SHARED → REGISTER LOAD

```
ldmatrix.sync.aligned.m8n8.x1.shared.b16 {d0}, [addr];
```

32 lanes 同步执行 · 1 个矩阵 · 每 lane 得到 1 × b32 (每个 b32 打包两个 b16)

.x1 .x2 .x4

Warp lanes

点击选择

T0-T7 → M0 的 8 条地址

0 1 2 3 4 5 6 7

T8-T15 只接收 fragment

8 9 10 11 12 13 14 15

T16-T23 只接收 fragment

16 17 18 19 20 21 22 23

T24-T31 只接收 fragment

24 25 26 27 28 29 30 31

该 lane 收到的 2 × b16

Shared Memory • 物理地址线

每条 16B 是一个逻辑行 (row-major)

M0 • 8×8 b16 → 每个 lane 的 d0

addr	c0	c1	c2	c3	c4	c5	c6	c7
addr0 T0	0,0 T0	0,1 T0	0,2 T1	0,3 T1	0,4 T2	0,5 T2	0,6 T3	0,7 T3
addr1 T1	1,0 T4	1,1 T4	1,2 T5	1,3 T5	1,4 T6	1,5 T6	1,6 T7	1,7 T7
addr2 T2	2,0 T8	2,1 T8	2,2 T9	2,3 T9	2,4 T10	2,5 T10	2,6 T11	2,7 T11
addr3 T3	3,0 T12	3,1 T12	3,2 T13	3,3 T13	3,4 T14	3,5 T14	3,6 T15	3,7 T15
addr4 T4	4,0 T16	4,1 T16	4,2 T17	4,3 T17	4,4 T18	4,5 T18	4,6 T19	4,7 T19
addr5 T5	5,0 T20	5,1 T20	5,2 T21	5,3 T21	5,4 T22	5,5 T22	5,6 T23	5,7 T23
addr6 T6	6,0 T24	6,1 T24	6,2 T25	6,3 T25	6,4 T26	6,5 T26	6,6 T27	6,7 T27
addr7 T7	7,0 T28	7,1 T28	7,2 T29	7,3 T29	7,4 T30	7,5 T30	7,6 T31	7,7 T31

Lane 的 register fragment

每个 d = 32 bit

T0 点击选中

目标寄存器 low 16 · high 16

d0 M0[0,0] [15:0] · b16 M0[0,1] [31:16] · b16

T0: 它还给出 M0 的行 0 起始地址; 与此同时, 作为结果 lane group T0-T3 的第 0 个 lane, 它从每个矩阵的同一条物理 line 0 收到 offset 0/1 两个 b16。

1 × (8 × 16B) → 32 × (1 × b32)

Bank Conflict

32 BANKS • 4 BYTES PER BANK WORD

$$\text{bank} = (\text{byte_addr} / 4) \bmod 32$$

一个 warp request 中，同一 bank 的不同 word 需要分批服务；最大 multiplicity 决定 conflict degree。

例外：多个 lane 读取同一个 word 可以 broadcast，不按“不同地址冲突”处理。

FP16 TILE • ROW STRIDE = 128 B • BASE 128B-ALIGNED

row	byte address	bank
0	base + 0 × 128	0
1	base + 1 × 128	0
2	base + 2 × 128	0
...	...	0
7	base + 7 × 128	0

128B 恰好跨过 32 个 banks 又回到 bank 0。跨 8 行读取相同 4B slot，会形成 8-way conflict。

ldmatrix 的 lane / register mapping 可以完全正确，但它所触碰的 SMEM 地址仍可能集中到同一组 banks。

SMEM Bank Conflict 与 XOR Swizzle

1 sector = 16B = 4 consecutive banks

访问 f16 • Sector column • 8×16B 索引 s0 s1 s2 s3 s4 s5 s6 s7

SMEM: 32 banks × 4B · $\text{physical_sector} = \text{logical_sector XOR row (base_offset=0)}$ · $\text{bank} = 4 \times \text{physical_sector} + \text{word_offset}$

Row-major (未 Swizzle)

同一 logical sector offset 落到同 4 banks

	physical s0	physical s1	physical s2	physical s3	physical s4	physical s5	physical s6	physical s7
r0	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7
r1	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7
r2	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7
r3	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7
r4	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7
r5	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7
r6	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7
r7	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7

本次请求命中的 32 Banks



8 service rounds • 8-way bank conflict

128B XOR Swizzle

每行的 sector 顺序按 row 置换

	physical s0	physical s1	physical s2	physical s3	physical s4	physical s5	physical s6	physical s7
r0	L0 s0	L1 s1	L2 s2	L3 s3	L4 s4	L5 s5	L6 s6	L7 s7
r1	L1 s0	L0 s1	L3 s2	L2 s3	L5 s4	L4 s5	L7 s6	L6 s7
r2	L2 s0	L3 s1	L0 s2	L1 s3	L6 s4	L7 s5	L4 s6	L5 s7
r3	L3 s0	L2 s1	L1 s2	L0 s3	L7 s4	L6 s5	L5 s6	L4 s7
r4	L4 s0	L5 s1	L6 s2	L7 s3	L0 s4	L1 s5	L2 s6	L3 s7
r5	L5 s0	L4 s1	L7 s2	L6 s3	L1 s4	L0 s5	L3 s6	L2 s7
r6	L6 s0	L7 s1	L4 s2	L5 s3	L2 s4	L3 s5	L0 s6	L1 s7
r7	L7 s0	L6 s1	L5 s2	L4 s3	L3 s4	L2 s5	L1 s6	L0 s7

本次请求命中的 32 Banks



1 service round • 无 bank conflict

逐 service round 查看 (每个 bank 每 round 服务一个 4B word) 全部 1 2 3 4 5 6 7 8

f16 Sector column 8×16B: 未 swizzle 时 8 行争用同 4 banks; XOR 后展开到全部 32 banks。

悬停 cell: 查看 logical → physical 映射。

三代 Tensor Core 的数据搬运

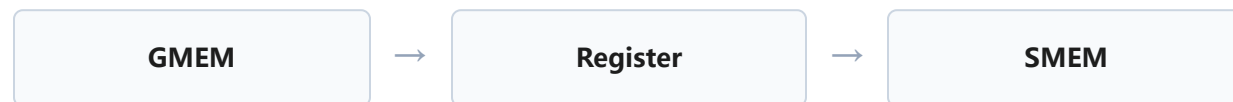


同样是“异步”，发起粒度、地址描述、消费者接口和完成协议都不同；不要把 `cp.async`、`TMA`、`WGMMA` 当作同一条指令的改名。

cp.async: 绕过中间 Register

SYNCHRONOUS THREAD COPY

load → register → store

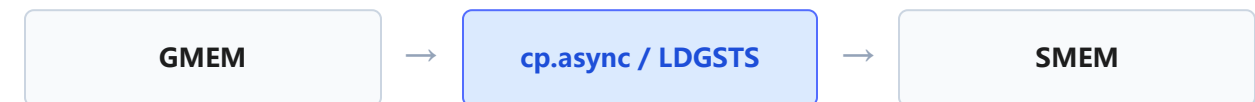


```
1 uint4 tmp = gmem[src];  
2 smem[dst] = tmp;  
3 __syncthreads();
```



SM80 ASYNC COPY • PER THREAD

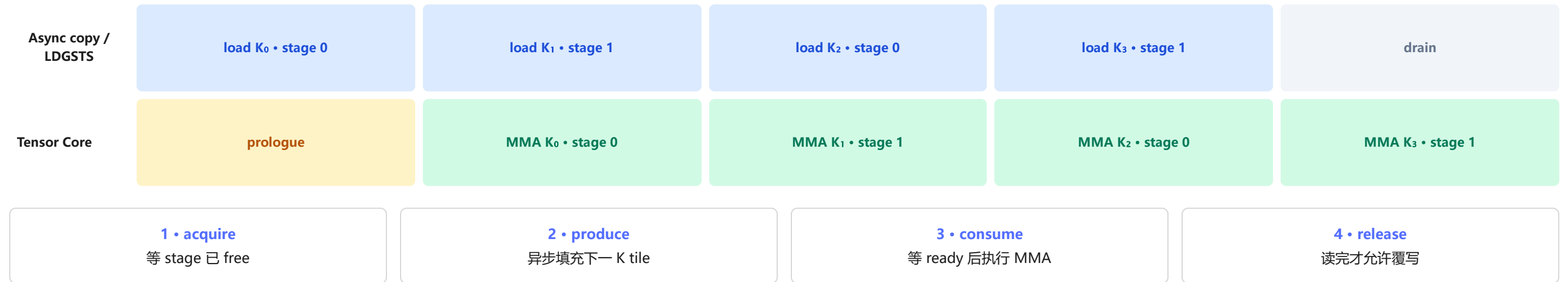
GMEM → SMEM



```
1 cp.async.cg.shared.global [dst], [src], 16;  
2 cp.async.commit_group;  
3 // compute on an older stage  
4 cp.async.wait_group 0;
```

cp.async 减少显式 load/store 指令与中间寄存器；单个 group 用 **wait_group 0** 才能证明 ready。多 stage 时，**wait_group N** 可保留至多 N 个最新 group 未完成。

Double Buffer: 空间换重叠



两个 stage 只提供“可以重叠”的存储空间；要真正重叠，还需要独立的 in-flight copy 与 compute，以及完整的 ready / free 协议。

TMA: 一个 Thread 异步搬运整个 Tile

TENSOR MEMORY ACCELERATOR

单条指令发起整个 Tile 的搬运



一个 elected thread 提交坐标；硬件执行多维地址生成、coalescing、可选 OOB fill 与 shared-memory swizzle。

OPAQUE TENSOR-MAP DESCRIPTOR

把规则放进 descriptor

base address	global shape
strides	box shape
element type	swizzle mode

```
1 cp.async.bulk.tensor.2d.shared::cta.global
2   .mbarrier::complete_tx::bytes
3   [smem], [tensor_map, {coord0, coord1}], [bar];
```


mbarrier 与 Warp Specialization

完成协议 • mbarrier

8-BYTE OBJECT IN SHARED MEMORY

当前世代
generation = 0,1,2,...

等待位
parity = generation & 1

未到达者
pending_count

TMA 未完成字节
tx_count

完成: pending_count == 0 且 tx_count == 0; 随后 generation 推进、parity 翻转。

1 • init

mbarrier.init(bar, 1)
初始化并建立可见性。

2 • expect

arrive.expect_tx(bytes)
登记本轮待完成 bytes。

3 • complete

TMA complete_tx(bytes)
硬件报告 copy 完成。

4 • wait

try_wait.parity(bar, p)
成功后 consumer 才读 SMEM。

控制流拆分 • Warp Specialization

TMA producer	load K ₀	load K ₁	load K ₂	load K ₃	next tile	next tile
MMA consumer	wait	MMA K ₀	MMA K ₁	MMA K ₂	MMA K ₃	signal
Writeback WG	previous tile	epilogue t-1	store t-1	wait	epilogue t	store t

Producer warp

tile schedule、TMA issue、barrier arrive。

MMA consumer

等待 ready，持续发异步 MMA，读完后发出 free。

Writeback WG

等待 accumulator，执行 epilogue / store。

TileLang Level-2

用 Tile Operations 表达 Tiled GEMM

Level-2 GEMM Kernel

一个 CTA 负责一个 128×256 的 C Tile; 沿 K 方向每次消费 64, 用 4-stage pipeline 重叠搬运与计算。

```
1 @tilelang.jit(target={"kind": "cuda", "arch": "sm_90a"})
2 def gemm_level2(A, B):
3     M, N, K = T.const("M, N, K")
4     A: T.Tensor((M, K), T.bfloat16)
5     B: T.Tensor((K, N), T.bfloat16)
6     C = T.empty((M, N), T.bfloat16)
7     BM, BN, BK = 128, 256, 64
8
9     with T.Kernel(N // BN, M // BM, threads=256) as (bx, by):
10         A_shared = T.alloc_shared((BM, BK), T.bfloat16)
11         B_shared = T.alloc_shared((BK, BN), T.bfloat16)
12         C_fragment = T.alloc_fragment((BM, BN), T.float32)
13         T.use_swizzle(panel_size=4)
14
15         T.clear(C_fragment)
16         for ko in T.Pipelined(K // BK, num_stages=4):
17             T.copy(A[by * BM, ko * BK], A_shared)
18             T.copy(B[ko * BK, bx * BN], B_shared)
19             T.gemm(A_shared, B_shared, C_fragment)
20
21         C_shared = T.alloc_shared((BM, BN), T.bfloat16)
22         T.copy(C_fragment, C_shared)
23         T.copy(C_shared, C[by * BM, bx * BN])
24
25     return C
```

T.copy 表达 Tile 搬运 • T.gemm 表达 Tile 计算 • T.Pipelined 表达跨 K Tile 的流水线

Summary

1 • SIMT Programming Model

Block 映射到 SM, Warp 共享指令流, Thread 保留独立状态; 优化时关注 coalescing 与 divergence。

2 • Tiled GEMM + Tensor Core

一个 Block 负责一个 C Tile, 沿 K 方向反复累加; 一个 Warp 或 WarpGroup 共同发起矩阵指令。

3 • Data Movement

GMEM → SMEM → Tensor Core 的路径需要布局、Swizzle、异步搬运、`mbarrier` 与流水线共同配合。

4 • TileLang Level-2

显式写出 Tile、存储位置和 `T.Pipelined` 数据流, 把常规 thread mapping 与指令 lowering 交给编译器。

高性能 GPU Kernel = 计算分块 + 数据布局 + 搬运流水线 + 正确同步